



AJAY KADIYALA - Data Engineer

Follow me Here:

LinkedIn:

<https://www.linkedin.com/in/ajay026/>

Data Geeks Community:

<https://lnkd.in/geknpM4i>

FAANG-Level Data Engineering Interview Kit

Table of Contents

1. How FAANG Interviews Work
2. SQL at FAANG Scale
3. Python for Data Engineers
4. System Design for Data Engineering (CORE FAANG ROUND)
5. Distributed Systems
6. Data Modeling & Warehousing
7. Cloud & Cost Optimization
8. Behavioral (FAANG Bar Raiser Round)
9. FAANG Scoring Rubric
10. FAANG Resume Bullets
11. Final FAANG Preparation Strategy
12. Top Questions & Answers asked in FAANG
13. FREE RESOURCES



1.1 How do you remove duplicates when data arrives late and order is not guaranteed?

Answer

duplicates are rarely clean. Data can arrive **late**, **out of order**, and **multiple times**. The first step is to define **what “duplicate” means** — usually a business key + event time.

A common approach is using ROW_NUMBER() with partitioning on the business key and ordering by event timestamp (or ingestion timestamp as fallback). We retain the latest or earliest record depending on business semantics.

However, FAANG interviewers expect you to **call out limitations**:

- Window functions are expensive on large datasets
- Late arrivals can change previous results
- Reprocessing/backfills may be required

In production, this is often combined with:

- Watermarks
- Incremental processing
- Idempotent writes

Follow-up they WILL ask

“What if data arrives 2 days late?”

You must explain **reprocessing strategy**, not just SQL.

1.2 Write a query to find session duration per user with a 30-minute inactivity window

Answer

This is a **sessionization** problem — extremely common at Google and Meta.

The idea is:

1. Sort events per user by timestamp
2. Identify session breaks when the gap exceeds 30 minutes
3. Assign session IDs using a cumulative sum
4. Aggregate per session

The core logic uses LAG() to compute time difference between events. When the gap exceeds 30 minutes, we flag a new session.

What FAANG evaluates:

- Can you think in steps?

- Do you understand temporal data?
- Can you explain logic clearly?

follow-up

“How does this perform on 5 billion rows?”

Expected answer: partitioning strategy, pre-sorting, and incremental computation.

1.3 How do you find Top-K records per group with ties?

Answer

This is NOT about LIMIT.

At FAANG level:

- You must use DENSE_RANK() instead of ROW_NUMBER()
- This ensures ties are preserved

Example use cases:

- Top products per category
- Top creators per region
- Top sessions per user

What matters is explaining **why** ROW_NUMBER() fails when ties exist.

follow-up

“What happens if K = 100 and groups are heavily skewed?”

You should mention:

- Data skew
- Memory pressure
- Possible pre-aggregation

1.4 How do you calculate running totals efficiently?

Answer

Running totals are classic **window function** problems.

At small scale:

`SUM(amount) OVER (PARTITION BY user_id ORDER BY date)`

At FAANG scale:

- Ordering is expensive
- Large partitions can spill to disk
- Performance degrades with skew

Interviewers expect you to mention:

- Pre-aggregation
- Bucketing by date
- Breaking computation into stages

Key insight

SQL correctness is easy.

SQL efficiency is what FAANG hires for.

1.5 How do you detect data skew in SQL queries?

Answer

Skew happens when a small number of keys dominate the data.

In SQL, skew can be detected by:

- Grouping counts per key
- Looking at max vs avg distribution
- Percentile-based analysis

Example reasoning:

If 5% of keys process 80% of data, joins and windows will bottleneck.

FAANG interviewers love candidates who **proactively detect skew**, not just react to failures.

Follow-up

“How would you fix skew?”

Expected: salting keys, repartitioning, or pre-aggregation.

1.6 Explain the difference between INNER JOIN and LEFT JOIN at scale

Answer

At logical level, difference is simple.

At FAANG scale, the **impact is massive**.

LEFT JOIN:

- Preserves unmatched records
- Can explode data size
- Often used incorrectly

INNER JOIN:

- Drops unmatched data
- Usually faster
- Safer for analytics unless business requires otherwise

FAANG expects you to **challenge requirements**:

“Do we really need a LEFT JOIN?”

This shows maturity.

1.7 How do you handle NULLs in aggregations correctly?

Answer

SQL handles NULLs inconsistently across functions.

Key points:

- COUNT(column) ignores NULLs
- COUNT(*) does not
- SUM(NULL) results in NULL unless handled

At FAANG scale, NULLs often represent:

- Missing data
- Late data
- Corrupted data

Correct handling requires:

- Explicit COALESCE
- Business-defined defaults
- Data quality checks upstream

Follow-up

“Should NULLs be fixed in SQL or ingestion?”

Correct answer: **as early as possible.**

1.8 How do you optimize a slow GROUP BY query?

Answer

FAANG expects a **systematic approach**, not guesses.

Steps:

1. Reduce input rows (filters early)
2. Reduce columns
3. Pre-aggregate where possible
4. Avoid high-cardinality group keys
5. Check skew

Wrong answer:

“Add more compute”

Right answer:

“Reduce work”

1.9 How do you compare two tables and find mismatched records?

Answer

This is common in reconciliation pipelines.

FAANG-level approaches:

- FULL OUTER JOIN with NULL checks
- EXCEPT / MINUS (engine dependent)
- Hash-based comparison for large tables

Interviewers care about:

- Accuracy
- Performance
- Explainability

Follow-up

“What if tables have billions of rows?”

Expected: partition-wise comparison and incremental checks.

1.10 How do you design SQL queries that are backfill-safe?

Answer

This is a **senior-level signal**.

Backfill-safe SQL means:

- Deterministic logic
- Idempotent outputs
- No dependency on load order
- Time-bound processing

FAANG values engineers who think:

“What happens when this reruns 6 months later?”

Most candidates don't. That's the separator.

Why this section is FAANG-grade

- Focuses on **thinking**, not syntax
- Includes **scale, skew, and cost**
- Mirrors **real interviewer follow-ups**

2.1 How would you process a 200GB CSV file using Python on a single machine?

Answer

Loading the file into memory is immediately ruled out. At FAANG, the expected approach is **streaming the file in chunks** using iterators or generators.

The key idea is:

- Read line by line or chunk by chunk
- Process each chunk independently
- Write intermediate results incrementally

This ensures:

- Constant memory usage
- Crash safety
- Scalability

Interviewers care more about **memory discipline** than syntax.

Follow-up

“What happens if the process crashes halfway?”

Expected answer: checkpointing or idempotent writes.

2.2 Why are generators preferred over lists in data pipelines?

Answer

Generators allow **lazy evaluation**. Data is produced only when needed.

At FAANG scale:

- Lists can cause memory spikes
- Generators enable streaming
- They fit naturally into ETL pipelines

A strong candidate explains:

“Generators convert memory problems into compute problems.”

That framing scores points.

2.3 How do you make a Python pipeline idempotent?

Answer

Idempotency means running the pipeline multiple times produces the **same result**.

Common strategies:

- Use deterministic keys
- Overwrite partitions instead of appending
- Track processed offsets or timestamps

FAANG interviewers want engineers who design pipelines assuming **retries will happen**.

Red flag answer

“We just rerun the job”

That signals inexperience.

2.4 How do you handle retries in Python without duplicating data?

Answer

Retries must be **safe by design**, not by luck.

Typical FAANG approach:

- Separate read, transform, write stages
- Commit only after successful write

- Use transactional or atomic operations

The candidate should explicitly say:

“Failure is normal. Duplication is not.”

2.5 Explain time complexity vs space complexity using a Python example

Answer

FAANG expects **trade-off awareness**.

Example reasoning:

- Using a dictionary increases memory but reduces lookup time
- Sorting reduces future computation but costs upfront

Good engineers explain **why they chose one**, not just what they chose.

2.6 How do you design a Python job that can resume after failure?

Answer

This is a senior-level question.

Expected components:

- Checkpoint state (file offset, batch ID, timestamp)
- Stateless transformations
- External state store (DB, file, metadata table)

FAANG interviewers are checking **production mindset**, not coding tricks.

2.7 When would you use multiprocessing vs multithreading in Python?

Answer

Correct answer mentions **GIL** without overexplaining it.

Key points:

- CPU-bound → multiprocessing

- I/O-bound → multithreading
- Overhead matters

A strong candidate adds:

“In data pipelines, I/O dominates more often than CPU.”

That shows real-world experience.

2.8 How do you validate data quality in Python pipelines?

Answer

Validation is not optional at FAANG.

Typical checks:

- Null checks
- Range validation
- Schema consistency
- Row counts

Important insight:

Validation failures should fail fast, not silently pass.

2.9 How do you handle schema changes in Python-based ingestion?

Answer

Schema drift is expected, not exceptional.

FAANG-friendly approaches:

- Schema inference with versioning
- Backward-compatible parsing
- Rejecting breaking changes

Interviewers want to hear:

“We design pipelines assuming schemas will change.”

2.10 What makes Python code production-grade at FAANG?

Answer

Not speed alone.

Production-grade Python means:

- Readable and modular
- Observable (logs, metrics)
- Testable
- Fail-safe

The best closing line:

“Anyone can write Python. FAANG hires engineers who can maintain it at scale.”

3.1 Design a large-scale event ingestion pipeline (Uber / Netflix scale)

Answer

I'd start by clarifying requirements: event volume, latency SLA, ordering guarantees, and retention. Typically, I design this as an **event-driven pipeline** with producers publishing events to a durable message bus.

The ingestion layer must absorb spikes, so I prefer a decoupled queue-based system. Downstream consumers handle validation, enrichment, and storage into a lakehouse or analytical store. Partitioning strategy is critical to avoid skew.

I explicitly design for **at-least-once ingestion** and handle duplicates downstream.

FAANG follow-up

“What breaks first at 10× traffic?”

Expected: broker saturation, hot partitions, downstream lag.

3.2 How would you design a real-time analytics pipeline with sub-minute latency?

Answer

The key is separating **ingestion**, **processing**, and **serving**.

I use a streaming engine to process events in near real time, apply windowed aggregations, and push results into a low-latency serving store. I keep raw events immutable for replay.

FAANG interviewers look for:

- Clear latency boundaries
- Windowing strategy
- Late data handling

I always explain **why batch fallbacks still exist**, even in real-time systems.

3.3 Design a CDC-based lakehouse ingestion system

Answer

CDC pipelines start with log-based capture from source databases. The key decision is whether CDC is **append-only** or **upsert-based** downstream.

I land raw CDC events first, then apply merge logic into curated tables. Schema evolution and reprocessing must be first-class citizens.

FAANG expects you to mention:

- Ordering guarantees
- Exactly-once semantics (or why not)
- Backfill strategy

Follow-up

“What if CDC lags for 2 hours?”

You must explain replay and reconciliation.

3.4 How do you design for late-arriving data?

Answer

Late data is normal at FAANG scale.

I design pipelines with:

- Watermarks

- Allowed lateness
- Recomputable aggregates

Critically, I explain **business impact**: dashboards may change, metrics may shift. Stakeholders must accept eventual consistency.

Strong candidates say:

“Late data is a product decision, not just a technical one.”

3.5 Design a fault-tolerant batch pipeline

Answer

Fault tolerance starts with **idempotency**.

I break the pipeline into stages with clear checkpoints. Each stage can be rerun independently. Outputs are written atomically or via overwrite semantics.

FAANG interviewers listen for:

- Clear failure domains
- Retry behavior
- Partial failure handling

3.6 How do you handle data skew in distributed systems?

Answer

Skew is one of the biggest real-world killers.

I detect skew early using distribution metrics. Solutions include salting keys, pre-aggregation, or isolating heavy keys into separate paths.

The important part is **admitting trade-offs**: salting complicates logic but improves performance.

3.7 Design a data pipeline with strong cost controls

Answer

At FAANG, cost is a feature.

I design pipelines to:

- Minimize reprocessing
- Use tiered storage
- Separate compute from storage

I always include **cost visibility**: per-job metrics, per-query costs, and alerts.

Follow-up

“How would you cut cost by 40%?”

Expected: reduce scans, pre-aggregate, adjust retention.

3.8 How do you design for schema evolution?

Answer

Schema changes are inevitable.

I design schemas to be backward compatible, versioned, and validated at ingestion.

Breaking changes are isolated behind new versions.

FAANG likes hearing:

“We never silently break consumers.”

3.9 How do you design a backfill strategy without breaking production?

Answer

Backfills are treated as **first-class workflows**.

I isolate backfill compute, throttle resource usage, and validate results before swapping.

Production pipelines remain unaffected.

Senior candidates emphasize:

- Reconciliation checks
- Controlled rollouts
- Clear rollback plans

3.10 How do you explain trade-offs during system design interviews?

Answer

FAANG doesn't expect a perfect system. They expect **clear reasoning**.

I always explain:

- Why I chose this design
- What I'm sacrificing
- When I'd redesign it

The strongest signal is saying:

"This works for today's scale, but I'd revisit it at 10× growth."

4.1 What is the difference between at-least-once and exactly-once processing?

Answer

At-least-once means a message may be processed more than once, especially after failures/retries. Exactly-once means each event affects the final result only once.

FAANG expects you to say the practical truth: "exactly-once" is usually achieved via a combination of:

- deduplication keys,
- idempotent writes,
- transactional sinks / commit logs,
not just a magic setting.

The key is business impact: for billing, duplicates are unacceptable; for dashboards, small duplicates might be tolerable.

Follow-up

"How would you achieve exactly-once in the sink?"

Answer should mention idempotency + transactional commit patterns.

4.2 What is backpressure and how do you handle it in streaming systems?

Answer

Backpressure happens when downstream processing is slower than ingestion, causing queues to build and latency to rise.

I handle it in layers:

- Scale consumers (horizontal)
- Tune batch sizes / micro-batch intervals
- Apply rate limiting
- Shed non-critical load (if allowed)

FAANG interviewers love when you explain you'd monitor:

- consumer lag,
- processing time vs arrival rate,
- watermark delay.

They want operational thinking, not textbook.

4.3 Why does partitioning matter so much in distributed processing?

Answer

Partitioning decides how work is split across machines. Bad partitioning causes:

- hot partitions (few nodes overloaded),
- slow jobs,
- memory spills.

FAANG expects you to connect partitioning to joins/windows:

- If partition key aligns with join key, shuffles reduce.
- If not, you pay heavy network + disk cost.

Strong answer: "Partitioning is performance design, not an implementation detail."

4.4 What is data skew, and how do you fix it?

Answer

Skew is uneven key distribution: a few keys hold most rows. It kills joins and window functions.

Fix strategies:

- Salting keys (split hot keys)
- Pre-aggregation before joins
- Isolating heavy keys into separate pipelines
- Custom partitioners (where supported)

FAANG wants you to mention trade-off: salting improves performance but complicates correctness and downstream merges.

4.5 What causes shuffles, and why are they expensive?

Answer

Shuffles happen when data must be redistributed across nodes, typically due to:

- joins,
- group by,
- order by,
- repartition.

They are expensive because they trigger:

- network IO,
- disk spill,
- serialization overhead,
- large task skew.

FAANG expects you to say: “Best optimization is to avoid shuffles by aligning partition keys and reducing data earlier.”

4.6 What is checkpointing and why is it critical?

Answer

Checkpointing stores progress/state so the system can recover after failure without restarting from scratch.

In streaming, it typically includes:

- offsets (what was read),
- state (window aggregates, session state),
- metadata for exactly-once.

Good answer also mentions risk:

- checkpoint corruption,
- too frequent checkpoints increase cost,
- too infrequent increases reprocessing.

This shows real production awareness.

4.7 How do you handle late-arriving events in streaming analytics?

Answer

Late data breaks “final” aggregates unless you plan for it.

I handle it using:

- watermarks to bound lateness,
- allowed lateness windows,
- correction mechanisms (upserts to serving layer),
- reprocessing from raw if needed.

FAANG expects you to make the product call explicit:

“Do we accept shifting metrics, or do we lock reports after a cutoff?”

That’s maturity.

4.8 What are the trade-offs between batch vs streaming for the same use case?

Answer

Streaming gives low latency but adds complexity: state, checkpoints, late data, replays.

Batch is simpler and cheaper, but delay is higher.

FAANG expects a balanced answer:

- Use streaming for operational decisions (fraud, alerts).
- Use batch for accuracy-heavy reporting and reprocessing.

Best answer includes hybrid:

“Streaming for fast insights + batch for correctness and backfills.”

4.9 How do you debug a distributed job that suddenly became 3× slower?

Answer

I approach it like an incident:

1. Check input size changes and skew
2. Compare stage times and shuffle sizes
3. Look for new joins or exploding transforms
4. Validate partitioning changes
5. Check cluster contention / resource limits

FAANG wants structured troubleshooting, not random tuning.

4.10 What reliability guarantees does Kafka provide in practice?

Answer

Kafka provides durability and ordering **within a partition**, not global ordering. Consumers track offsets; failures can lead to reprocessing unless commits are handled carefully.

FAANG expects you to explicitly mention:

- ordering = per partition,
- delivery = at-least-once by default,

- exactly-once needs proper producer + transactional write pattern.

If you say “Kafka is exactly-once,” they’ll push back.

5.1 How do you decide between event-based and snapshot-based modeling?

Answer

Event-based models store every action as it happens; snapshot-based models store state at a point in time.

At FAANG scale, I prefer **event-based modeling** for flexibility and replayability. It allows reprocessing when logic changes and supports multiple downstream use cases.

Snapshots are still useful for:

- fast reporting,
- point-in-time analysis,
- cost-efficient dashboards.

Strong candidates explain **both** and choose based on use case, not habit.

5.2 Star schema vs Snowflake schema — how do you choose?

Answer

Star schemas favor performance and simplicity. Snowflake schemas reduce redundancy but increase join complexity.

FAANG expects you to say:

- Star schema for high-performance analytics
- Snowflake when dimensions are large, shared, or frequently changing

The real signal:

“I optimize for query patterns, not modeling purity.”

5.3 How do you design a fact table for long-term scalability?

Answer

I start by defining the **grain** clearly — one row per what?

Then:

- Keep fact tables narrow
- Avoid derived columns when possible
- Store immutable facts
- Use surrogate keys for joins

FAANG interviewers listen for clarity on grain. Ambiguity here is a red flag.

5.4 What is Slowly Changing Dimension (SCD) and when do you use it?

Answer

SCD handles changes in dimensional attributes over time.

At FAANG scale:

- Type 1 for corrections
- Type 2 when history matters (pricing, org structure)
- Type 3 rarely, for limited comparisons

Strong candidates add:

“Not every dimension deserves SCD Type 2 — it increases storage and complexity.”

That judgment matters.

5.5 How do you handle schema evolution in analytical tables?

Answer

Schema evolution is inevitable.

I design for:

- backward compatibility,
- additive changes first,
- versioned schemas for breaking changes.

FAANG wants to hear:

“Consumers should never break silently.”

This shows ownership beyond ingestion.

5.6 How do you model time correctly in data warehouses?

Answer

Time modeling is tricky and often mishandled.

I separate:

- event time,
- processing time,
- reporting date.

I also avoid overloading timestamps with business logic. Clear time semantics prevent subtle bugs in analytics.

FAANG interviewers care deeply about time correctness.

5.7 How do you design models that support backfills safely?

Answer

Backfill-safe models are:

- deterministic,
- idempotent,
- time-partitioned.

I ensure rerunning historical data produces identical results and doesn't double-count.

Senior candidates explicitly say:

“Every model should assume it will be rebuilt.”

5.8 How do you balance performance vs flexibility in warehouse design?

Answer

Performance comes from pre-aggregation; flexibility comes from raw detail.

FAANG-friendly design:

- Store raw, immutable data
- Build curated, aggregated layers
- Allow recomputation when logic changes

This layered approach avoids locking into early assumptions.

5.9 How do you validate data correctness in warehouse models?

Answer

I validate at multiple levels:

- row counts,
- uniqueness constraints,
- null checks,
- metric reconciliation.

FAANG expects validation to be **automated**, not manual. Silent data errors are worse than job failures.

5.10 What modeling mistakes cause pain at FAANG scale?

Answer

Common mistakes:

- unclear grain,
- mixing snapshot and event logic,
- over-denormalization,
- embedding business logic too early.

The best closing line in interviews:

“Most data problems are modeling problems, not tooling problems.”

6.1 How do you design data pipelines with cost in mind from Day 1?

Answer

Cost-aware design starts before writing code. I first understand data volume, access patterns, and SLAs. Then I separate **storage and compute**, so each can scale independently.

I avoid full reprocessing by designing incremental pipelines and time-based partitions. I also define clear data retention rules—raw data lives longer, derived data expires sooner.

FAANG interviewers look for this line:

“The cheapest query is the one you never run.”

6.2 How do you identify cost bottlenecks in an existing data platform?

Answer

I don't start by guessing. I start by **measuring**.

I look at:

- scan volume per query,
- compute hours per pipeline,
- top expensive dashboards,
- reprocessing frequency.

Often, the biggest cost drivers are repeated full-table scans and unnecessary joins. FAANG expects structured analysis, not “let's add caching.”

6.3 How would you reduce analytics cost by 40% without breaking SLAs?

Answer

I attack cost in layers:

1. Reduce scanned data (partition pruning, column pruning)

2. Pre-aggregate hot metrics
3. Cache commonly accessed results
4. Adjust refresh frequency where real-time isn't required

Strong candidates explicitly say:

“Not every dashboard needs minute-level freshness.”

That product-awareness matters.

6.4 Compute-heavy vs storage-heavy workloads — how do you design differently?

Answer

For compute-heavy workloads, I optimize execution plans, reduce shuffles, and right-size clusters.

For storage-heavy workloads, I focus on:

- compression,
- tiered storage,
- lifecycle policies.

FAANG expects you to say:

“Optimization depends on where the money is going.”

6.5 How do you decide retention policies for raw vs curated data?

Answer

Raw data is cheap to store and expensive to regenerate, so I retain it longer. Curated and aggregated data is cheap to rebuild but expensive to store at scale.

Typical approach:

- raw: long retention,
- intermediate: medium,
- aggregates: short, business-driven.

FAANG interviewers want to hear **intentional retention**, not defaults.

6.6 How do you control cost in multi-tenant data platforms?

Answer

Multi-tenancy requires isolation.

I implement:

- workload-level quotas,
- cost attribution by team or product,
- alerts on abnormal usage.

Strong answer:

“Teams should feel the cost of their queries.”

This aligns incentives and drives responsible usage.

6.7 How do you handle cost surprises after a new feature launch?

Answer

Cost spikes usually come from unexpected access patterns.

I respond by:

- identifying new query paths,
- adding targeted aggregates,
- throttling or caching abusive queries,
- aligning product expectations with cost reality.

FAANG values engineers who **own outcomes**, not just pipelines.

6.8 How do you balance performance optimization vs cost optimization?

Answer

They often conflict.

I prioritize:

- correctness first,

- then SLA,
- then cost.

If performance gains cost disproportionately more, I push back with data. FAANG respects engineers who challenge requirements with facts.

6.9 How do you design for cost observability?

Answer

Cost should be observable like latency or errors.

I expose:

- per-job cost,
- per-query cost,
- trend dashboards,
- anomaly alerts.

Strong candidates say:

“If you can’t see cost, you can’t control it.”

6.10 What cost-related mistakes do engineers commonly make?

Answer

Common mistakes:

- over-refreshing data,
- storing redundant aggregates,
- ignoring skew-driven cost,
- scaling compute instead of fixing design.

The strongest closing line:

“Most cloud cost problems are data modeling problems in disguise.”

7.1 Tell me about a time a critical data pipeline failed in production

Answer

I describe the incident clearly, without blaming systems or people. I focus on **what broke, why it broke, and how I responded**.

I explain how I prioritized restoration first, then root-cause analysis. After recovery, I implemented preventive controls—alerts, validation checks, and runbooks.

FAANG looks for:

- calm under pressure
- ownership
- learning mindset

Strong closing:

“The incident improved our reliability long-term.”

7.2 Describe a time you disagreed with a PM or stakeholder

Answer

I frame it as a difference in goals, not conflict. I explain how I used data—cost, latency, correctness—to explain trade-offs.

Even if my proposal wasn’t chosen, I show alignment and execution support.

FAANG values:

- respectful disagreement
- data-backed decisions
- collaboration over ego

7.3 Tell me about a decision that involved a difficult trade-off

Answer

I explain the constraints: time, cost, correctness, scale. I clearly state **what I sacrificed and why**.

FAANG interviewers love hearing:

“This was not perfect, but it was right for that moment.”

That shows maturity.

7.4 Describe a time you improved an inefficient system

Answer

I explain the baseline, how I measured inefficiency, and the changes I made. I quantify impact: cost reduced, latency improved, failures avoided.

FAANG wants measurable outcomes, not vague improvements.

7.5 Tell me about a time you handled data quality issues

Answer

I show that I treated data quality as a **product issue**, not just a bug.

I explain:

- how issues were detected,
- how stakeholders were informed,
- how guardrails were added.

Strong candidates say:

“We made data quality visible, not hidden.”

7.6 Describe a time you had to deliver under tight deadlines

Answer

I focus on prioritization. What did I **cut**, **defer**, or **simplify** to meet the deadline?

FAANG values engineers who know what *not* to build.

7.7 Tell me about a time you mentored or influenced others

Answer

This signals **senior/staff readiness**.

I explain how I shared context, reviewed designs, or raised team standards. Impact matters more than title.

Good line:

“The team scaled, not just the system.”

7.8 Describe a time you made a mistake

Answer

Never say “I don’t make mistakes.”

I own the mistake, explain the impact, and—most importantly—what changed afterward. FAANG looks for accountability and growth.

7.9 Tell me about a time you handled ambiguous requirements

Answer

I explain how I asked clarifying questions, proposed assumptions, and validated them early.

FAANG prefers engineers who **reduce ambiguity for others**.

7.10 Why should we hire you as a data engineer?

Answer

I don’t repeat my resume.

I summarize:

- how I think about data systems,
- how I balance correctness, scale, and cost,
- how I take ownership beyond my tasks.

Strong ending:

“I build systems that survive growth, failure, and change.”

8.1 What FAANG means by “Strong Hire”

- Clear problem framing
- Correctness under edge cases
- Explicit trade-offs
- Ownership mindset
- Ability to scale the design mentally

Key signal: interviewer asks *follow-ups* because they’re interested.

8.2 What “Hire” vs “Leaning Hire” looks like

- **Hire:** solid solution + minor gaps
- **Leaning Hire:** good ideas, weak depth or shaky execution

Most candidates sit here. This is where Bar Raiser decides.

8.3 Common reasons strong candidates still get rejected

- Over-indexing on tools
- No cost awareness
- Ignoring failure modes
- Vague behavioral answers
- Not challenging requirements

FAANG rejects *unsafe hires*, not weak resumes.

8.4 How interview feedback is actually written

Typical feedback themes:

- “Good SQL, weak system thinking”

- “Strong ownership, needs depth”
- “Technically solid, but unclear trade-offs”

Understanding this helps you **aim your answers**.

8.5 How to recover if an answer goes wrong

FAANG values recovery.

Strong move:

“Let me restate with a better assumption.”

Defensive move = red flag.

SECTION 9 — FAANG Resume Bullets

FAANG resumes are **impact documents**, not skill lists.

9.1 Weak vs Strong Resume Bullet

Weak:

Built Spark pipelines to process data

Strong:

Designed Spark-based ingestion pipeline processing 5B+ events/day, reducing latency by 35% and compute cost by 28%

9.2 What FAANG looks for in bullets

- Scale (rows, TBs, QPS)
- Impact (cost, latency, reliability)
- Ownership (designed, led, migrated)
- Outcome (measurable)

9.3 How many bullets per role?

- 3–5 strong bullets
- Each bullet should survive “**So what?**”

9.4 Common resume mistakes

- Tool dumping
- No numbers
- Vague verbs
- Mixing responsibilities with impact

FAANG resumes are **signal-dense**, not long.

9.5 Resume alignment with interview answers

Your resume bullets should map directly to:

- system design stories
- behavioral examples
- trade-offs you discuss

Consistency builds trust.

ALAN SUSA

Data Engineer

alansusa@email.com

(123) 456-7890

New York, NY

WORK EXPERIENCE

Data Engineer

Consumer Reports

- May 2018 - current New York, NY
 - Led the migration from Oracle to Redshift using Amazon Athena and S3, resulting in an annual cost savings of \$678,000 and an increase in performance of 14%.
 - Designed and implemented a real-time data pipeline to process semi-structured data by integrating 150 million raw records from 30+ data sources using Kafka and PySpark.
 - Designed the data pipeline architecture for a new product that quickly scaled from 0 to 125,000 daily active users.
 - Studied and revamped data dictionaries to include a more robust history for developing consistency across domain.

Data Engineer

Guardian Life Insurance Company

- August 2016 - May 2018 New York, NY
 - Maintained data pipeline up-time of 99.8% while ingesting streaming and transactional data across 8 different primary data sources using Spark, Redshift, S3, and Python.
 - Automated ETL processes across billions of rows of data, which reduced manual workload by 25% monthly.
 - Ingested data from disparate data sources using a combination of SQL, Google Analytics API, and Salesforce API using Python to create data views to be used in BI tools like Tableau.
 - Communicated with project managers and analysts about data pipelines that drove efficiency KPIs up by 20%.

Data Engineer Intern

Federal Reserve Board of Governors

- August 2014 - August 2016 Washington, DC
 - Built basic ETL that ingested transactional and event data from a web app with 12,000 daily active users that saved over \$85,000 annually in external vendor costs.
 - Worked with client to understand business needs and translate those business needs into actionable reports in Tableau, saving 17 hours of manual work each week.
 - Used Spark in Python to distribute data processing on large streaming datasets, improving ingestion and speed by 67%.
 - Supported implementation and active monitoring of controls and programs for precision and efficacy.

EDUCATION

B.A.

Computer Science

University of Pittsburgh

September 2010 - April 2014

Pittsburgh, PA

SKILLS

- Python
- ETLs
- SQL (Postgres, Redshift, MySQL)
- NoSQL (MongoDB)
- Spark, Kafka
- Airflow
- AWS (Athena, Lambda, S3)



Jane Roe

Business Development Manager

jane.roe@gmail.com

202-555-0166

New York, USA

linkedin.com/in/jane.roe

jane.roe

Professional Business Developer with more than four years of experience in the business development processes. Involved in product testing, management, and development of new business opportunities.

WORK EXPERIENCE

Business Development Manager

AirState Solutions of

New York, USA

- Successfully managed \$2 - 3 million budget projects and successfully achieved the project scheduled goals.
- Developed and implemented new marketing and sales plans and defined the strategy for the next 5 years.
- Reviewed constantly the customer feedback and then suggested ways to improve the processes and customer service levels which increased the satisfaction rate from 81% to 95%.
- Ensured that new clients will grow into a loyal customer base in a specialist niche market by implementing a new loyalty program.

Business Development Assistant

AirState Solutions

Chicago, USA

- Increased the customer satisfaction rate by 25% by improving the customer service.
- Planned, supervised, and coordinated daily activity of 3 junior business analysts.
- Improved the communication with the Marketing department to better understand the competitive position.
- Directed the creation and implementation of a Business Continuity Plan, and the management of audit programs.

EDUCATION

MSc in Economics and Business Administration

The University of Chicago

09/2008 - 06/2010

SKILLS

- SEO
- Public Speaking
- Negotiation
- Teamwork
- Decision Making
- Research & Strategy
- Emotional Intelligence
- Outbound Marketing
- Email Marketing
- Google Analytics
- Sales & Marketing

ORGANIZATIONS

American Management Association (2015 - Present)

Association of Private Enterprise Education (2014 - Present)

eBusiness Association (eBA) (2013 - Present)

HONOURS AND AWARDS

Jury Member, Venture Cup Entrepreneurship Competition (2016)

Venture Cup USA

Sales Individual & Business Development Award (2015)

AirState Business Awards

Excellence in Customer Partnering Award (IES - Institute of Excellence in Sales)

LANGUAGES

English

Spanish

French

Ajay Kadiyala

Senior Data Engineer

📍 India | ✉️ ajay@email.com | 🔗 [LinkedIn](#) | 🔗 [GitHub](#)

SUMMARY

Senior Data Engineer with **7+ years** of experience designing and scaling large-scale data platforms. Specialized in **batch & streaming pipelines, lakehouse architectures, distributed systems, and cost optimization**. Proven track record of owning data systems handling **billions of records** with strong reliability, performance, and cost discipline.

CORE SKILLS

- Data Engineering: Batch & Streaming Pipelines, CDC, Lakehouse
- SQL: Advanced Analytics, Window Functions, Optimization
- Programming: Python, PySpark
- Distributed Systems: Kafka, Spark, Airflow
- Cloud: AWS / GCP / Azure (vendor-agnostic mindset)
- Data Modeling: Event-based, Dimensional, SCD
- Soft Skills: Ownership, System Design, Mentorship

PROFESSIONAL EXPERIENCE

Senior Data Engineer

Global Tech Company (FAANG-like scale) | 2021 – Present

- Designed and owned a **real-time event ingestion pipeline** processing **5B+ events/day**, reducing downstream latency by **42%**.
- Built **CDC-based lakehouse architecture** enabling reprocessing and schema evolution without breaking consumers.
- Optimized Spark pipelines by fixing data skew and shuffle bottlenecks, cutting compute cost by **35%**.

- Led data modeling for analytics platform supporting **50+ dashboards** with consistent metrics and backfill safety.
- Implemented cost observability dashboards, enabling teams to attribute and reduce query costs proactively.
- Mentored junior engineers and reviewed system designs to raise team-wide data engineering standards.

Data Engineer

Enterprise Client / Consulting Firm | 2018 – 2021

- Developed batch and streaming pipelines using Spark and Kafka for high-volume transactional data.
- Implemented SQL-based validation frameworks to catch data quality issues before downstream consumption.
- Designed star-schema and event-based models supporting historical and real-time analytics.
- Collaborated with product and analytics teams to translate business requirements into scalable data solutions.

SYSTEM DESIGN EXPERIENCE

- Designed scalable ingestion systems with **at-least-once processing and idempotent sinks**
- Built hybrid **streaming + batch** architectures for accuracy and replayability
- Designed cost-efficient pipelines using incremental processing and partition pruning

SELECTED IMPACT METRICS

-  5B+ events/day processed
-  35–40% cloud cost reduction
-  40% latency improvement

-  Zero-downtime backfills

EDUCATION

Bachelor's Degree in Engineering

(Details omitted intentionally – FAANG prioritizes experience)



30-Day **FAANG** Data Engineer Interview Prep Plan

Structured plan to ace your *FAANG* interview



WEEK 1: FOUNDATIONS

- ✓ Advanced SQL (Window Functions)
- ✓ Python & Memory Management
- ✓ Core Data Modeling Concepts

WEEK 2: SYSTEM DESIGN

- ✓ 1 Design/Day Practice
- ✓ Explain Trade-Offs Aloud
- ✓ Scale & Failure Handling

WEEK 3: DISTRIBUTED & CLOUD

- ✓ Skew & Shuffles
- ✓ Cost Optimization Tactics
- ✓ Batch vs Streaming Scenarios

WEEK 4: BEHAVIORAL PRACTICE

- ✓ Prepare STAR+ Stories
- ✓ Mock Interviews
- ✓ Refine Answers

FINAL 2 DAYS: READY

- ✓ Review Key Frameworks
- ✓ No New Topics
- ✓ Stay Calm & Focused

Focus on clarity, not perfection.

SECTION 10 — 30-Day FAANG Interview Preparation Plan

Week 1: Foundations

- Deep SQL (windowing, sessionization)
- Python streaming & memory patterns
- Revise core data modeling concepts

Week 2: System Design

- 1 design per day
- Practice explaining trade-offs aloud
- Focus on scale + failure + cost

Week 3: Distributed Systems + Cloud

- Skew, shuffles, backpressure
- Cost optimization scenarios
- Batch vs streaming decisions

Week 4: Behavioral + Mock Rounds

- Prepare 8–10 STAR+ stories
- Do mock interviews
- Refine explanations (not solutions)

Final 48 Hours

- No new topics
- Revise frameworks

- Practice calm, structured explanations

Strong closing mindset:

“I don’t need perfect answers. I need clear thinking.”

Top 10 Data Engineering Interview Questions -Google

1. How would you design a data pipeline to process billions of events per day?

Answer (What Google expects)

I’d start by clarifying **event volume, latency requirements, and failure tolerance**. For billions of events, the pipeline must be **event-driven and horizontally scalable**.

I’d decouple ingestion from processing using a durable message layer. Processing would be distributed, stateless where possible, and designed for **at-least-once delivery**, with idempotent writes downstream. Raw data would always be stored immutably to allow replay.

Google cares less about tools and more about:

- decoupling,
- backpressure handling,
- replayability.

Key line that scores points:

“I assume failures are normal and design for reprocessing.”

2. How do you handle late-arriving data in analytics systems?

Answer

Late data is expected at Google scale. I design pipelines with **event-time semantics**, not processing time.

I use:

- watermarks to bound lateness,
- allowed lateness windows,

- recomputable aggregates.

I also align with stakeholders on **metric mutability**—some dashboards may change after initial publication.

Strong signal:

“Late data handling is a product decision, not just a technical one.”

3. Explain how you would detect and handle data skew

Answer

I detect skew by analyzing key distributions—max vs avg, percentiles, and per-partition metrics.

To fix skew:

- I pre-aggregate heavy keys,
- apply key salting where correctness allows,
- or isolate hot keys into separate processing paths.

Google expects you to **acknowledge trade-offs**:

“Salting improves performance but complicates downstream aggregation.”

4. How do you ensure data quality in large-scale pipelines?

Answer

Data quality must be enforced **early and automatically**.

I implement:

- schema validation at ingestion,
- null and range checks,
- volume and freshness checks,
- metric reconciliation.

Failures should **fail fast** and alert owners. Silent corruption is worse than downtime.

Google likes hearing:

“We treat data quality as a first-class system, not a downstream fix.”

5. How would you design a backfill-safe pipeline?

Answer

A backfill-safe pipeline is:

- deterministic,
- idempotent,
- time-partitioned.

I ensure rerunning historical data produces identical results. Backfills run in isolated environments and are validated before swapping outputs.

Senior-level signal:

“Every pipeline should assume it will be rebuilt.”

6. Explain batch vs streaming — when would you choose each?

Answer

Streaming is ideal for low-latency decisions like alerts or fraud detection. Batch is better for correctness-heavy analytics and reprocessing.

At Google, I often use **hybrid systems**:

- streaming for quick insights,
- batch for accuracy and backfills.

Best framing:

“Streaming gives speed; batch gives truth.”

7. How do you optimize a slow SQL query on very large datasets?

Answer

I approach it systematically:

1. Reduce scanned data (filters, partitions)

2. Reduce columns
3. Avoid unnecessary joins
4. Check for skew
5. Pre-aggregate where possible

Google wants **methodical debugging**, not random tuning.

8. How do you handle schema evolution in data pipelines?

Answer

Schema changes are inevitable.

I design pipelines to be:

- backward compatible by default,
- versioned for breaking changes,
- validated before consumption.

Strong signal:

“Consumers should never break silently due to schema changes.”

9. Describe a time you had to make a difficult trade-off in a data system

Answer

I explain the constraints clearly—time, cost, scale, correctness. I describe what I chose, what I sacrificed, and why.

Google values engineers who can say:

“This wasn’t perfect, but it was the right decision given the constraints.”

10. How do you think about cost optimization in data platforms?

Answer

Cost optimization starts with **visibility**.

I monitor:

- scan volume,
- compute usage,
- reprocessing frequency.

I reduce cost by:

- incremental processing,
- pre-aggregation,
- lowering refresh frequency where possible.

Strong closing line:

“The cheapest pipeline is the one that doesn’t reprocess unnecessarily.”

Top 10 Data Engineering Interview Questions -Amazon

1. How would you design a data pipeline that processes billions of records daily with strict SLAs?

Answer

I start by clarifying SLAs, retry tolerance, and downstream consumers. At Amazon scale, I design pipelines to be **incremental, idempotent, and observable**.

I decouple ingestion from processing, ensure retries don’t create duplicates, and design outputs to be partitioned by time or business keys. I also build alerts around SLA breaches, not just job failures.

Amazon signal line:

“I design assuming retries and partial failures will happen.”

2. How do you ensure your data pipelines are cost-efficient?

Answer

Cost is treated as a first-class metric.

I track:

- compute hours per pipeline,
- data scanned per query,
- frequency of reprocessing.

Then I optimize by:

- switching from full to incremental loads,
- pruning partitions and columns,
- reducing refresh frequency for non-critical data.

Strong Amazon framing:

“Cost efficiency is part of ownership, not an afterthought.”

3. Tell me about a time a pipeline failed and how you handled it

Answer

I explain the failure calmly: what broke, how I detected it, and how I restored service quickly.

Then I focus on **long-term fixes**—alerts, validation, and runbooks—so the issue doesn’t repeat.

Amazon interviewers listen for:

- accountability,
- bias for action,
- learning.

Avoid blaming systems or people.

4. How do you design pipelines to be backfill-safe?

Answer

I design pipelines to be deterministic and idempotent. Outputs are written in overwrite or merge mode, never blind appends.

Backfills are isolated from production runs and validated before promotion.

Senior-level signal:

“Backfills should be routine, not risky events.”

5. How do you handle schema changes from upstream systems?

Answer

I assume schemas will change.

I validate schemas at ingestion, allow additive changes by default, and version breaking changes. Downstream consumers are protected from sudden breaks.

Amazon values engineers who **protect customers of data**, not just pipelines.

6. How do you deal with data quality issues discovered late?

Answer

I treat data quality issues as incidents.

First, I assess business impact. Then I backfill corrected data and communicate changes clearly to stakeholders. Finally, I add upstream checks to prevent recurrence.

Amazon mindset:

“Fix the root cause, not just the symptom.”

7. How do you optimize a slow-running ETL job?

Answer

I don't tune blindly.

I:

1. Check input size and skew
2. Review joins and aggregations
3. Reduce data early
4. Verify partitioning strategy

If needed, I refactor logic instead of just scaling compute.

Amazon prefers **deep dives**, not surface fixes.

8. How do you balance speed of delivery vs long-term maintainability?

Answer

I'm explicit about trade-offs.

For urgent needs, I deliver a simple, correct solution and document limitations. I schedule follow-ups to harden the system.

Strong answer:

"Fast doesn't mean careless; it means deliberate trade-offs."

9. How do you design data systems that multiple teams depend on?

Answer

I design for stability and clear contracts.

That means:

- documented schemas,
- versioned changes,
- deprecation paths,
- clear ownership.

Amazon values engineers who think in terms of **internal customers**.

10. Why should Amazon trust you to own critical data pipelines?

Answer

I explain that I think end-to-end: ingestion, correctness, cost, failures, and customer impact.

I emphasize ownership:

"If it breaks at 3 AM, I know exactly how to fix it and how to prevent it next time."

That aligns perfectly with Amazon's bar.

Top 10 Data Engineering Interview Questions -Meta

1. How would you design a real-time analytics pipeline for user events?

Answer

I'd clarify latency targets, event volume, and correctness requirements. The pipeline should be **event-time based**, decoupled at ingestion, and capable of handling spikes.

I keep raw events immutable for replay, compute near-real-time aggregates for fast insights, and allow corrections via late-data handling.

Meta signal line:

"We optimize for fast insights first, then converge to correctness."

2. How do you handle late-arriving events in real-time dashboards?

Answer

Late data is expected at Meta scale. I use watermarks with allowed lateness and design aggregates to be **mutable** for a defined window.

I also align with product teams on how much metric drift is acceptable after initial publish.

Strong framing:

"Metric stability is a product decision, not just a streaming setting."

3. How would you design data pipelines to support A/B experiments?

Answer

Experiments require **clean assignment, isolation, and reproducibility**.

I ensure:

- deterministic user bucketing,
- immutable event logs,
- clear experiment metadata,
- backfill capability when logic changes.

Meta interviewers check whether you understand **experiment correctness**, not just data flow.

4. How do you ensure metrics consistency across teams?

Answer

I centralize metric definitions and expose them via shared, versioned models. Teams consume standardized metrics instead of redefining logic.

Strong candidates say:

“We reduce metric debates by making definitions explicit and shared.”

5. How do you deal with schema evolution in fast-moving products?

Answer

I assume schemas change frequently.

I design ingestion to tolerate additive changes, validate breaking changes, and version schemas when needed. Downstream consumers are protected from silent breaks.

Meta cares about speed + safety, not one or the other.

6. How do you detect and fix data quality issues quickly?

Answer

I rely on automated checks for freshness, volume, and distribution shifts. When issues arise, I triage impact first, then fix upstream causes.

Meta values:

- fast detection,
- fast communication,
- fast iteration.

7. How do you handle data skew in real-time systems?

Answer

I monitor per-key and per-partition load continuously. For hot keys, I apply salting or split processing paths.

I also evaluate whether the skew is **temporary (events, launches)** or structural, and fix accordingly.

8. How would you backfill data without breaking live dashboards?

Answer

Backfills run in isolation and update aggregates gradually. I validate results before swapping and communicate expected metric changes.

Senior signal:

“Backfills should be boring, not scary.”

9. How do you balance speed vs correctness at Meta?

Answer

I bias toward speed for product iteration, but always keep a path to correctness.

Streaming gives fast feedback; batch reconciles and corrects. This hybrid approach keeps teams moving without sacrificing trust.

10. Why should Meta trust you with user-critical data systems?

Answer

I explain that I design systems that are fast, observable, and correctable. I own failures end-to-end and optimize for product impact, not just technical elegance.

Strong closing:

“I help teams move fast without breaking trust in data.”

Top 10 Data Engineering Interview Questions -Netflix

1. How would you design a data pipeline to track video playback events at global scale?

Answer

I'd start with clear requirements: event volume, latency, and data consumers (product, recommendations, reliability). I'd design an **event-driven ingestion** with durable buffering to absorb spikes, store raw events immutably for replay, and compute curated aggregates for analytics.

Key design choices include partitioning by user/session, handling duplicates via idempotent sinks, and ensuring backpressure doesn't cascade.

Netflix signal line:

"We optimize for correctness first, because bad data leads to bad product decisions."

2. How do you ensure data accuracy in streaming analytics?

Answer

Accuracy comes from **event-time processing**, consistent schemas, and validation. I design streaming aggregates to be mutable within a defined window and reconcile with batch backfills for final correctness.

I also publish **data confidence indicators** so consumers know when metrics are provisional vs final.

3. How would you handle late-arriving playback events?

Answer

Late events are normal due to network variability. I use watermarks and allowed lateness, and I design aggregates that can be corrected within a defined SLA.

Netflix expects you to align on **how long metrics are allowed to change** and communicate that clearly to stakeholders.

4. How do you design data systems that support experimentation (A/B tests)?

Answer

I ensure deterministic experiment assignment, immutable raw events, and consistent aggregation logic across control and treatment. I also design pipelines to support backfills when experiment logic changes.

Strong framing:

“Experiments are only as good as the data correctness behind them.”

5. How do you detect and prevent data quality regressions?

Answer

I implement automated checks for freshness, volume, distribution shifts, and schema changes. Alerts are tied to **business impact**, not just technical failures.

Netflix values proactive detection over reactive fixes.

6. How do you handle schema evolution in event data?

Answer

I expect frequent changes. I design schemas to be backward compatible, validate breaking changes early, and version events when necessary. Downstream consumers are insulated from sudden breaks.

7. How do you design pipelines that are easy to backfill?

Answer

Backfillability is designed in:

- deterministic logic,
- time-partitioned data,
- idempotent writes.

Backfills run in isolation, are validated, and then promoted safely.

Senior signal:

“Reprocessing should be routine, not risky.”

8. How do you balance latency vs correctness in analytics?

Answer

I separate **fast-path** metrics (near real-time) from **accurate-path** metrics (batch-corrected). Consumers understand which to use for which decisions.

Netflix prefers clarity over pretending one number fits all needs.

9. How do you reason about cost in large-scale data pipelines?

Answer

I track compute and storage usage per pipeline and per consumer. I reduce cost by minimizing reprocessing, pre-aggregating hot metrics, and enforcing retention policies.

Key line:

“Cost efficiency enables us to run more experiments, not fewer.”

10. Why should Netflix trust your data systems?

Answer

I explain that I design systems that are observable, correctable, and aligned with decision-making. I prioritize trust in data, because at Netflix, data directly influences product and content decisions.

Strong close:

“Reliable data enables better storytelling and better products.”

Top 10 Data Engineering Interview Questions – Apple

1. How would you design a data pipeline that prioritizes user privacy?

Answer

I start by minimizing data collection—only what’s strictly required. I design pipelines with **data anonymization**, **aggregation early**, and **strict access controls**. Sensitive attributes are handled separately with strong governance.

Strong Apple signal:

“Privacy is a design constraint, not a feature added later.”

2. How do you ensure data correctness across multiple downstream consumers?

Answer

I establish clear data contracts: schema definitions, SLAs, and versioning. I centralize critical transformations so logic isn't reimplemented differently across teams.

Apple values predictability:

“The same input should always produce the same output.”

3. How do you handle schema changes without breaking existing reports?

Answer

I prefer backward-compatible changes. For breaking changes, I version schemas and provide a deprecation window. Consumers migrate deliberately, not accidentally.

Apple expects **controlled change**, not rapid churn.

4. How do you design pipelines that remain stable for years?

Answer

I optimize for simplicity and determinism:

- fewer moving parts,
- clear ownership,
- strong monitoring,
- documented assumptions.

Apple prefers systems that evolve slowly but safely.

5. How do you validate data quality in production systems?

Answer

I implement automated checks for freshness, volume, and invariants. I tie alerts to business impact and maintain dashboards that show data health over time.

Strong framing:

“If data quality degrades silently, the system has already failed.”

6. How do you handle late or missing data?

Answer

I design with event time and allow bounded lateness. Missing data is surfaced explicitly rather than hidden. Consumers know when data is partial.

Apple values **explicitness over cleverness**.

7. How do you think about performance optimization vs reliability?

Answer

I prioritize reliability first. Optimizations that risk correctness or debuggability are avoided. Performance improvements are incremental and measurable.

Apple prefers:

“Slightly slower and correct over fast and fragile.”

8. How do you design data systems that scale globally?

Answer

I design region-aware ingestion, local processing where possible, and centralized aggregation when required. I handle time zones, localization, and regulatory constraints explicitly.

Apple expects global thinking from day one.

9. Tell me about a time you prevented a data issue before it reached users

Answer

I describe early detection—via validation or anomaly checks—and proactive communication. The focus is on prevention and learning, not firefighting.

Apple values quiet success.

10. Why should Apple trust you with long-lived data systems?

Answer

I explain that I design systems to be predictable, privacy-aware, and maintainable. I optimize for long-term stability and user trust over short-term wins.

Strong close:

“Reliable data systems protect the user experience.”

Top 10 Data Engineering Interview Questions -Microsoft

1. How would you design a data platform used by multiple enterprise teams?

Answer

I start with **clear data contracts**—schemas, SLAs, ownership, and versioning. I separate raw, curated, and serving layers so teams can innovate without breaking each other. Access is role-based, and changes follow a deprecation process.

Microsoft signal line:

“Platforms succeed when teams can move independently without breaking trust.”

2. How do you enforce data governance at scale?

Answer

Governance must be built-in, not manual. I enforce schema validation, lineage tracking, access controls, and audit logs. Sensitive data is tagged and restricted by default.

Microsoft expects governance to be **automated and visible**, not policy-only.

3. How do you design for backward compatibility in data systems?

Answer

I design schemas to be additive by default. For breaking changes, I version outputs and provide a migration window. Consumers never wake up to broken dashboards.

Strong framing:

“Backward compatibility protects downstream velocity.”

4. How do you handle data quality across dozens of downstream consumers?

Answer

I centralize critical transformations and expose shared metrics. Automated checks validate freshness, volume, and invariants. Issues are surfaced early with clear ownership.

Microsoft values **predictable data** over fast but inconsistent data.

5. How would you migrate a large on-prem data warehouse to the cloud?

Answer

I migrate in phases: replicate data first, validate parity, then cut over consumers gradually. I keep old and new systems running in parallel until confidence is high.

Key signal:

“Migrations fail when parity and rollback aren’t planned.”

6. How do you design pipelines to be resilient to partial failures?

Answer

I break pipelines into isolated stages with checkpoints. Each stage is idempotent and rerunnable. Failures are contained and don’t cascade.

Microsoft interviewers listen for **clear failure domains**.

7. How do you think about cost management in enterprise data platforms?

Answer

I attribute cost by workload and team. I set budgets, alerts, and usage dashboards so teams see the impact of their queries.

Strong line:

“Cost transparency changes behavior.”

8. How do you manage schema evolution across multiple teams?

Answer

I enforce schema registries, versioning, and compatibility checks. Breaking changes require explicit approval and communication.

Microsoft expects **process-backed engineering**, not ad-hoc fixes.

9. How do you ensure reliability during peak enterprise usage?

Answer

I design for peak from day one: capacity planning, load testing, and graceful degradation. Non-critical workloads are throttled first.

Reliability is measured in **customer trust**, not uptime alone.

10. Why should Microsoft trust you to build shared data platforms?

Answer

I explain that I design systems that scale with teams, enforce governance automatically, and remain stable through growth and change.

Strong close:

“Great platforms make the right thing easy and the wrong thing hard.”

Top 10 Data Engineering Interview Questions -Uber

1. How would you design a real-time pipeline for ride events (requests, trips, cancellations)?

Answer

I'd design an **event-driven, low-latency pipeline** with strong decoupling. Producers emit immutable ride events; consumers handle enrichment and aggregation. Partitioning is critical—typically by city or geo-hash—to localize load and reduce skew.

I design for **at-least-once processing** with idempotent sinks and keep raw events for replay.

Uber signal line:

“I design assuming traffic spikes during peak hours and events.”

2. How do you handle geo-based data skew (e.g., a busy city center)?

Answer

Geo-skew is inevitable. I detect it via per-partition metrics and mitigate it by:

- finer-grained geo-partitioning,
- salting hot geo-keys,
- isolating mega-hot regions into dedicated pipelines.

Uber interviewers expect you to acknowledge **dynamic skew**—it changes by time of day.

3. How would you compute real-time ETAs or surge-related metrics?

Answer

I separate fast-path computation from accurate-path reconciliation. Streaming computes near-real-time metrics; batch jobs reconcile for accuracy and backfills.

State is kept minimal and checkpointed. I bound lateness explicitly to avoid unbounded state growth.

Key framing:

“Speed first, correctness soon after.”

4. How do you ensure correctness in real-time billing-related data?

Answer

Billing requires stronger guarantees. I design with idempotent writes, deterministic keys, and reconciliation jobs. Any discrepancy triggers alerts and compensating corrections.

Uber expects engineers to be conservative with money-impacting data.

5. How do you handle late or out-of-order events in streaming systems?

Answer

I use event time with watermarks and allowed lateness. Aggregates are mutable within a defined window; after that, corrections go through controlled backfills.

Strong candidates mention **business-aligned lateness windows**, not arbitrary settings.

6. How do you design pipelines to survive partial regional outages?

Answer

I design region-aware ingestion with failover paths. Critical data is buffered durably; non-critical processing can degrade gracefully.

Uber values systems that **degrade safely**, not ones that crash loudly.

7. How do you debug a real-time pipeline that suddenly lags during peak hours?

Answer

I check ingestion rate vs processing rate, consumer lag, and hot partitions. I compare partition load by geo/time to identify skew or downstream saturation.

Uber wants **structured, on-call-ready thinking**.

8. How do you backfill historical ride data without affecting live systems?

Answer

Backfills run in isolated environments with throttled resources. Results are validated before merging. Live pipelines remain unaffected.

Senior signal:

“Backfills should never compete with live traffic.”

9. How do you balance latency vs cost in real-time systems?

Answer

I tune latency to business needs. Not every metric needs sub-second freshness. I reduce cost by batching where possible and pre-aggregating hot paths.

Uber respects engineers who challenge unnecessary latency requirements.

10. Why should Uber trust you with real-time, city-scale data systems?

Answer

I explain that I design systems that handle spikes, geo-skew, and failures gracefully. I optimize for operational reliability because real-time data directly affects riders and drivers.

Strong close:

“Reliable real-time data keeps the city moving.”

Top 10 Data Engineering Interview Questions -Airbnb

1. How would you design a data pipeline for a two-sided marketplace (hosts & guests)?

Answer

I start by modeling **events separately** for hosts and guests, with a shared marketplace context (listing, booking, location). I keep raw events immutable and build curated models for demand, supply, and conversion.

Key is **clear grain** and **consistent joins** so metrics don't drift across teams.

Airbnb signal line:

“Marketplace metrics must be comparable across time and experiments.”

2. How do you design data systems to support A/B experiments safely?

Answer

I ensure deterministic user bucketing, immutable event logs, and versioned metric definitions. Experiment metadata is joined centrally so control/treatment logic isn't reimplemented differently.

Strong candidates say:

"Experiments fail when assignment or metrics aren't reproducible."

3. How do you ensure metric consistency across teams?

Answer

I centralize metric definitions and expose them via shared models. Teams consume metrics, not raw tables. Changes are versioned and communicated.

Airbnb values **one definition of truth**, even if iteration is slower.

4. How would you handle late-arriving booking or cancellation events?

Answer

I use event-time processing with allowed lateness and design aggregates to be mutable within a defined window. For financial or trust metrics, I reconcile with batch backfills.

Key framing:

"Late data is expected; uncorrected data is unacceptable."

5. How do you design pipelines for Trust & Safety analytics?

Answer

I design separate, stricter pipelines for sensitive signals. Access is tightly controlled, transformations are audited, and outputs are explainable.

Airbnb expects **privacy, correctness, and traceability**—not black boxes.

6. How do you detect data quality issues that impact business decisions?

Answer

I monitor freshness, volume, and distribution shifts, and I reconcile key metrics against historical baselines. Alerts are tied to business KPIs.

Airbnb prefers early detection over post-hoc explanations.

7. How do you backfill marketplace data without breaking dashboards?

Answer

Backfills run in isolation, results are validated, and swaps are done atomically. I communicate expected metric changes in advance.

Senior signal:

“Metric movement is okay; surprises are not.”

8. How do you balance speed of iteration vs data trust?

Answer

I allow fast iteration on raw and exploratory layers, but curated metrics move slower with stronger validation.

Airbnb likes this balance:

“Move fast on ideas, move carefully on metrics.”

9. How do you think about cost optimization in analytics platforms?

Answer

I reduce cost by minimizing reprocessing, pre-aggregating hot metrics, and adjusting refresh frequency. I attribute cost by team to drive responsible usage.

Cost efficiency enables more experimentation.

10. Why should Airbnb trust you with marketplace-critical data systems?

Answer

I explain that I design systems that are reproducible, trustworthy, and aligned with business decisions. I prioritize data clarity because it directly affects host and guest experience.

Strong close:

“Trust in data builds trust in the marketplace.”

Top 10 Data Engineering Interview Questions -LinkedIn

1. How would you design a data pipeline for a large professional graph (members, connections, interactions)?

Answer

I model interactions as **events** (views, messages, connections) and keep the **graph state derived**, not primary. Raw events remain immutable for replay. Curated graph features (degrees, mutuals, embeddings) are computed incrementally.

Signal line:

“Graphs are derived views; events are the source of truth.”

2. How do you compute graph metrics (e.g., mutual connections) at scale?

Answer

I avoid full recomputation. I use **incremental updates** driven by edge changes and cache hot results. Heavy recomputes run in batch; low-latency needs use precomputed features.

LinkedIn expects awareness of **compute explosion** in graph joins.

3. How do you design pipelines for personalization features?

Answer

I separate **offline training features** from **online serving features** but keep them **definition-aligned**. Feature generation is versioned and reproducible to prevent training/serving skew.

Strong framing:

“Feature correctness matters more than feature count.”

4. How do you ensure data quality for metrics used by leadership?

Answer

I centralize definitions, add freshness/volume checks, and reconcile metrics across sources. Changes are versioned and communicated.

LinkedIn values **stable executive metrics** over rapid churn.

5. How do you handle late or out-of-order events in engagement analytics?

Answer

I use event time with bounded lateness. Aggregates are mutable within a window; final numbers are batch-reconciled.

Key line:

“Early metrics guide action; final metrics guide strategy.”

6. How do you manage schema evolution across many producer teams?

Answer

I enforce schema registries with compatibility checks. Additive changes are allowed; breaking changes require versioning and deprecation windows.

LinkedIn prefers **platform guardrails** over manual policing.

7. How do you backfill features without impacting live personalization systems?

Answer

Backfills run in isolated compute with throttles. Results are validated and swapped atomically. Live systems consume only validated versions.

Senior signal:

“Feature backfills should never surprise online systems.”

8. How do you balance experimentation speed with data trust?

Answer

Exploration happens on raw layers; trusted metrics live in curated layers with stricter checks. Experiments move fast, but reporting moves carefully.

9. How do you think about cost optimization in large analytics platforms?

Answer

I reduce cost via incremental processing, pre-aggregation of hot metrics, and right-sized refresh rates. I expose cost by team to align incentives.

10. Why should LinkedIn trust you with member data systems?

Answer

I explain that I design **reproducible, privacy-aware, and scalable** systems. I optimize for signal quality because it directly affects member experience.

Strong close:

“High-quality data builds better professional connections.”

Top 10 Data Engineering Interview Questions -Stripe

1. How would you design a data pipeline for payment transactions?

Answer

I design with **immutability and auditability** at the core. Every transaction event is written once and never mutated. Corrections are handled via compensating events, not updates.

I separate ingestion, validation, and accounting views. Downstream aggregates are derived, reproducible, and traceable back to raw events.

Stripe signal line:

“Financial data should be explainable down to a single event.”

2. How do you guarantee idempotency in data pipelines?

Answer

Idempotency is non-negotiable.

I use deterministic keys (transaction ID + version), deduplicate at the sink, and ensure retries don't create duplicates. Writes are atomic and either fully succeed or fail.

Stripe expects you to say:

"Retries are expected; duplicates are not."

3. How do you handle late or out-of-order payment events?

Answer

I process by **event time**, not arrival time. Late events are accepted within defined bounds and reconciled via compensating records.

For finalized financial reports, I lock periods and apply corrections separately with full audit trails.

Key framing:

"Financial correctness beats freshness."

4. How do you design pipelines that are audit-friendly?

Answer

I ensure every derived number can be traced back to raw inputs. I store metadata—source, timestamp, version—and maintain lineage.

Stripe values systems where an auditor can ask:

"Why is this number here?"

...and get a clear answer.

5. How do you ensure data quality in money-related systems?

Answer

I implement strict validation: schema, value ranges, invariants (e.g., debits = credits). Failures stop the pipeline immediately and alert owners.

Silent data corruption is unacceptable.

6. How do you design backfills for financial data?

Answer

Backfills run in isolation and produce new versions, never overwrite historical truth. Results are reconciled before being exposed.

Stripe prefers **append-only correction models** over destructive rewrites.

7. How do you handle schema evolution in financial events?

Answer

Schemas are versioned and backward compatible. Breaking changes require new versions, not silent updates. Consumers migrate explicitly.

Strong signal:

“Financial schemas evolve slowly and deliberately.”

8. How do you balance performance vs correctness in payment analytics?

Answer

Correctness always wins. I optimize performance only after correctness is guaranteed. Any optimization must preserve determinism and auditability.

Stripe values engineers who **refuse risky shortcuts**.

9. How do you detect and respond to data inconsistencies?

Answer

I reconcile aggregates against raw events and historical baselines. Any mismatch triggers investigation and rollback paths.

Fast detection limits blast radius.

10. Why should Stripe trust you with financial data systems?

Answer

I explain that I design systems that are deterministic, auditable, and resilient to failure. I treat financial data as a ledger, not just analytics.

Strong close:

“Trust in payments starts with trust in data.”

FREE RESOURCES

◆ SYSTEM DESIGN (FAANG CORE ROUND)

1 Google Engineering – System Design Docs

<https://github.com/donnemartin/system-design-primer>

Why it's gold:

This is the *most trusted* free system design repo. Covers scalability, trade-offs, diagrams, and follow-ups.

2 High Scalability (Real-world architectures)

<http://highscalability.com>

Why FAANG loves this:

Real architectures used by Netflix, Uber, Twitter. Great for *thinking like a staff engineer*.

3 Data Engineering System Design (Free GitHub)

<https://github.com/InterviewReady/system-design-resources>

Use it for:

End-to-end data pipelines, batch vs streaming, lakehouse patterns.

◆ SQL (FAANG DEPTH, NOT BASIC)

4 SQL Interview Questions – Mode Analytics

<https://mode.com/sql-tutorial>

Why it's excellent:

Business-focused SQL with real datasets. FAANG-style thinking.

5 LeetCode SQL (Free Section)

<https://leetcode.com/problemset/database/>

Use it for:

Window functions, joins, ranking. Don't memorize—understand patterns.

6 SQLZoo

<https://sqlzoo.net>

Why it works:

Hands-on SQL with increasing complexity. Still very relevant.

♦ PYTHON (DATA ENGINEERING STYLE)

7 Python for Data Engineering (Free Book)

<https://github.com/jakevdp/PythonDataScienceHandbook>

Why FAANG-aligned:

Focuses on efficiency, data handling, not scripting.

8 Real Python – Free Tutorials

<https://realpython.com>

Use it for:

Generators, memory, iterators, production patterns.

♦ DISTRIBUTED SYSTEMS (HIDDEN FILTER)

9 Designing Data-Intensive Applications (Free Notes)

 <https://github.com/ms2ag16/Books/blob/master/Designing%20Data-Intensive%20Applications.pdf>

Why this matters:

This book is *practically the FAANG bible* for data systems.

10 Apache Spark Internals (Free)

 <https://spark.apache.org/docs/latest/>

Use it for:

Shuffles, partitioning, execution model—very common follow-ups.

◆ **STREAMING / REAL-TIME**

1 1 Kafka Definitive Guide (Free online sections)

 <https://developer.confluent.io/learn-kafka/>

Why FAANG loves it:

Exactly-once, offsets, ordering—asked everywhere.

1 2 Streaming Systems (Google – Free Talks)

 <https://www.youtube.com/watch?v=Z3tcmG3nA4E>

Use it for:

Event time, watermarks, late data (Google/Meta favorite).

◆ **MOCK INTERVIEWS (FREE)**

1 3 Pramp (Mock Interviews – FREE)

 <https://www.pramp.com>

Why it's great:

Peer-to-peer FAANG-style mock interviews. Very realistic.

1 4 Interviewing.io – Free Prep Content

 <https://interviewing.io/blog>

Use it for:

Real FAANG interview experiences and expectations.

◆ **REAL FAANG QUESTIONS & EXPERIENCES**

1 5 **Glassdoor Interview Experiences**

 <https://www.glassdoor.com/Interview>

DataGeeks